

Origin

Origin is the combination of protocol (scheme), host (domain) and port number.

So basically: Origin = Protocol + Host + Port

Consider the URL: <https://www.google.com:443>

Here, 'https' is the Protocol, 'www.google.com' is the host and '443' is the port number.

If any of the three (protocol, host, port) is different, then two URLs are considered to be different origins. Even if one of them is a subdomain, it's a different origin.

Also, the presence of `www` and absence of it makes the URLs to be different origins.

Let's take a few examples:

1. `http://mywebsite.com` and `http://www.mywebsite.com` are different origins due to different hosts.
2. `http://mywebsite.com` and `https://mywebsite.com` are different origins due to different protocols.
3. `http://mywebsite.com` and `http://mywebsite.com:8080` are different origins due to different ports.

Same-Origin Policy (SOP)

This policy is a security mechanism enforced by browsers. According to this policy, if a webpage running on origin A makes a request to access a resource that exists on origin B, then the browser will allow the request but block access to the response due to SOP. However, if the resource that the webpage is trying to access exists on the same origin, then it is allowed by the browser.

Same-origin policy does not interfere with sending request to the server on origin B, however the response that comes from the server will not be accessible by the webpage running on origin A. The access to the response will be denied by the browser because of the Same-Origin Policy of the browser.

For example, if a script on `http://mywebsite.com` tries to fetch data from `http://anotherwebsite.com`, the browser will allow the request to be sent but will block the script from accessing the response due to SOP.

Disadvantage of SOP

Although this policy has its own security benefits by isolating potentially malicious websites, it can be overly restrictive.

Let's suppose that we have a website that depends on a third-party payment gateway for handling payments. In this case, because the two web applications exist on different origins, they will not be able to communicate because of SOP.

That's why CORS (Cross-Origin Resource Sharing) was introduced.

CORS

CORS is a mechanism that allows servers to specify who can access their resources from different origins, thus providing some relaxation on the SOP restrictions.

This is achieved by HTTP headers. So basically, whenever a request comes to the server, it will process the request and, in the response, includes CORS headers like 'Access-Control-Allow-Origin', which basically tells the browser whether to let the requesting webpage access the response, or deny to read the response.

That is CORS in at a high-level.

Now, before we move on to how CORS works, we need to know about CORS headers.

How CORS works

1. Let's suppose we are trying to send a request from Origin A to Origin B.
2. If it's a simple request (GET or POST requests with certain safe headers), then the request is sent directly to the server on origin B. The server processes the request and includes CORS headers in the response. The response is then sent back to the browser. Now, we have the following possibilities:
 - i. If the information mentioned in the CORS headers indicates that it's alright for origin A to access the response, then the browser allows the webpage on origin A to read the response.
For example: If the 'Access-Control-Allow-Origin' header matches with the Origin A, then the browser will allow origin A to access the response.
 - ii. If the information mentioned in the CORS headers indicates it's not alright for origin A to read the response, or the CORS headers do not exist in the HTTP header, then the browser blocks the response from being read by the requesting origin A.
3. If it is a complex request (methods other than GET or POST or those with custom headers), the browser first sends a 'pre-flight request' using OPTIONS method to the server of origin B instead of sending the actual request directly. Pre-flight requests job is simply to determine whether it's safe to send the actual request or not. This pre-flight request contains the information about the actual request to let server decide if it's a safe request. When the server responds to the pre-flight request then we have following possibilities:
 - i. If the pre-flight response headers say it's okay to send the actual request, then the browser will send the actual request.
Now, the server of origin B will process the actual request and send the response (including the CORS headers) back to the browser. The CORS header is again checked by the browser to see if the requesting origin A is allowed to read the response.
 - ii. If the pre-flight response says it's not OK to send the actual request, then, the actual request is blocked.

HTTP MESSAGES

HTTP (Hypertext Transfer Protocol) messages are the fundamental units of data exchange between a client and a server. These messages consist of:

1. **Request Messages:** Sent from the client to the server, requesting a resource or action.
2. **Response Messages:** Sent from the server to the client, containing the requested resource or status information about the request.

HTTP Request Message Structure

An HTTP request message is composed of several parts:

1. **Request Line:** Contains the method, URL, and HTTP version.

```
GET /index.html HTTP/1.1
```

2. **Headers:** Contain metadata about the request.

```
Host: www.example.com  
User-Agent: Mozilla/5.0  
Accept: text/html
```

3. **Blank Line:** Separates headers from the body.
4. **Body:** Optional part that contains data sent to the server (e.g., form data, JSON payload).

```
name=John&age=30
```

HTTP Response Message Structure

An HTTP response message is composed of several parts:

1. **Status Line:** Contains the HTTP version, status code, and reason phrase.

```
HTTP/1.1 200 OK
```

2. **Headers:** Contain metadata about the response.

```
Content-Type: text/html  
Content-Length: 138
```

3. **Blank Line:** Separates headers from the body.
4. **Body:** Contains the actual data (e.g., HTML page, JSON payload).

HTTP Headers

HTTP headers are key-value pairs that are included in both request and response messages. They provide essential information about the request or response, such as content type, content length, authentication credentials, and caching policies. Headers are crucial for the communication between the client and the server as they convey important details that affect how the messages are processed.

CORS HEADERS

Pre-flight Request Headers

1. **Access-Control-Request-Method:** The HTTP method (e.g., PUT, DELETE) of the actual request.
2. **Access-Control-Request-Headers:** A comma-separated list of additional headers that will be included in the actual request.
3. **Origin:** The origin of the requesting web page.

```
OPTIONS /api/resource HTTP/1.1
Host: api.example.com
Origin: https://example.com
Access-Control-Request-Method: PUT
Access-Control-Request-Headers: X-Custom-Header
```

Pre-flight Response contains

The server responds to the preflight request with headers that specify which origins, methods, and headers are allowed:

1. **Access-Control-Allow-Origin:** Specifies the origin(s) that are allowed to access the resource.
2. **Access-Control-Allow-Methods:** Lists the HTTP methods that are allowed when accessing the resource.
3. **Access-Control-Allow-Headers:** Lists the headers that are allowed when accessing the resource.
4. **Access-Control-Allow-Credentials** (optional): Indicates whether the response to the request can be exposed when credentials are included in the request (for authenticated pages).
5. **Access-Control-Max-Age** (optional): Indicates how long the results of a preflight request can be cached.

```
HTTP/1.1 200 OK
Access-Control-Allow-Origin: https://example.com
Access-Control-Allow-Methods: GET, POST, PUT
Access-Control-Allow-Headers: X-Custom-Header, Content-Type
Access-Control-Allow-Credentials: true
Access-Control-Max-Age: 86400
```

Server Response Headers

1. **Access-Control-Allow-Origin:**
 - a. Specifies which origin is allowed to access the resource.
 - b. Can be a specific origin (e.g., https://example.com) or a wildcard (*).
2. **Access-Control-Allow-Headers:** Lists the headers that the server permits in the actual request.
3. **Access-Control-Allow-Credentials:** Indicates whether the response can be exposed when credentials (such as cookies) are included.
4. **Access-Control-Expose-Headers:** Lists the headers that are safe to expose to the requesting origin.

CSRF

CSRF is a type of attack where a malicious site tricks a user's browser into making unwanted actions on another site where the user is authenticated.

The attack relies on the fact that browsers automatically include session cookies (or other credentials like basic authentication headers, bearer tokens, etc.) with requests to the origin that set them. This is true only for form submissions.

Form Submissions (CSRF Context)

- **Same-Origin Requests:** Cookies are automatically included.
- **Cross-Origin Requests:** Cookies are automatically included by the browser. This is what makes CSRF possible because a malicious site can submit a form to another site where the user is authenticated, and the browser includes the cookies of the target site automatically.

JavaScript Requests (XHR/Fetch)

- **Same-Origin Requests:** Cookies are automatically included by default.
- **Cross-Origin Requests:** Cookies are not included by default. You need to explicitly configure the request to include cookies, and the server must support it.

CSRF Attack Scenario:

1. User Logs into Bank.com:

- The user navigates to <https://bank.com> and logs in using their credentials.
- After logging in, bank.com sets a session cookie in the user's browser to maintain the session.

2. User Visits a Malicious Site (Cute-Cats.com):

- The user, while still logged into bank.com, opens a new tab and visits <https://cute-cats.com>, which is controlled by an attacker.

3. Malicious Site Initiates CSRF Attack:

- cute-cats.com contains hidden HTML forms or scripts that automatically submit requests to <https://bank.com>.
- For instance, the site might include a hidden form like this:

```
<form id="csrf-form" action="https://bank.com/transfer"
method="POST" style="display:none;">

  <input type="hidden" name="amount" value="1000">

  <input type="hidden" name="to_account" value="attacker_account">
</form>

<script>

  document.getElementById('csrf-form').submit();
</script>
```

This form is set to transfer money from the user's account to an account controlled by the attacker.

4. Request Sent to Bank.com:

- When the hidden form is submitted, the browser sends a request to <https://bank.com/transfer>.
- Because the user is logged into bank.com, the browser includes the session cookie with the request to authenticate the user.

5. Bank.com Processes the Request:

- bank.com receives the request along with the session cookie.
- The server checks the session cookie, sees that it is valid, and assumes the request is legitimate and made by the authenticated user.
- Consequently, bank.com processes the money transfer request.

6. Response Handling:

- The server processes the request and transfers the money.
- The response from bank.com (e.g., a confirmation of the transfer) is sent back to the browser.

- Because of CORS, the browser will block the response from being accessed by scripts on cute-cats.com, but this is irrelevant to the attacker.
- The attack's goal—to transfer money—has already been achieved by the time the server responds.

Since the browser sends the session cookie with the request, the server needs to have measures in place to verify that the request is legitimate, such as CSRF tokens or SameSite cookies.